AI Classical Systems: Simple Search

---

1. Introduction to Search in AI

- Search is a **fundamental technique** in AI for problem-solving.
- Most AI problems can be framed as **finding a path** from an initial state to a goal state in a **problem space**.
- **Simple search algorithms** systematically explore this problem space by expanding nodes (states) until the goal is found.
- The **problem space** is represented as a graph or tree:
    - **Nodes:** States of the problem.
    - **Edges:** Operators or actions that transform one state to another.

---

2. Elements of a Search Problem

Before applying search, a problem must be defined formally with these components:

Component	Description
Initial State	The state where the search begins.
Goal State	The desired final state or condition to be achieved.

State Space	The set of all possible states reachable via valid operations.
Operators	Actions or rules that move from one state to another (edges in the graph).
Path Cost	Numeric cost associated with a path (optional for simple search).

---

3. Simple Search Overview

- **Simple search algorithms** perform **uninformed search** (or blind search).
- They explore the problem space without any domain-specific knowledge or heuristics.
- Search strategies differ in the order they expand nodes.
- The goal is to find **any path** from initial state to goal state (not necessarily optimal).

---

4. Types of Simple Search

A) Depth-First Search (DFS)

- Explores as deep as possible along each branch before backtracking.
- Uses a **stack** (LIFO) structure (often implemented via recursion).
- **Procedure:**
    - Start from initial node.

    - Explore one child, then that child's child, and so on.
    - If dead-end or goal not found, backtrack and explore siblings.
- **Advantages:**
    - Requires less memory (stores only current path and unexplored siblings).
    - Simple to implement.
- **Disadvantages:**
    - Can get stuck in deep or infinite paths.
    - Not guaranteed to find the shortest path.
- **Example:** Exploring a maze by always going as far as possible before backtracking.

B) Breadth-First Search (BFS)

- Explores all nodes at the current depth before moving to the next level.
- Uses a **queue** (FIFO) structure.
- **Procedure:**
    - Start from initial node.
    - Expand all immediate successors.
    - Then expand successors of successors, and so on.
- **Advantages:**
    - Finds the shortest path (fewest steps) if path cost is uniform.
    - Complete: will find a solution if one exists.

- **Disadvantages:**
    - Requires large memory (stores all nodes at current level).
    - Can be slow for deep or large state spaces.
- **Example:** Searching a family tree level by level.

C) Uniform Cost Search (UCS) (also called Dijkstra's Algorithm)

- Expands the node with the lowest total path cost from the start.
- Uses a **priority queue** ordered by path cost.
- **Advantages:**
    - Finds least-cost path, even if path costs differ.
    - Complete and optimal (if costs are positive).
- **Disadvantages:**
    - Can be expensive in time and memory.
- **Example:** Finding the cheapest route on a map.

D) Other Simple Search Variants

- **Iterative Deepening DFS (IDDFS):** Combines DFS's low memory with BFS's completeness by performing DFS with increasing depth limits.
- **Bidirectional Search:** Searches simultaneously from start and goal states and meets in the middle, potentially reducing search time.

---

5. Detailed Working of DFS and BFS

Aspect	Depth-First Search (DFS)	Breadth-First Search (BFS)
Data Structure	Stack (explicit or recursion)	Queue
Order of Node Expansion	Deep along one branch first, backtrack if needed	Level by level (all nodes at depth d before d+1)
Memory Usage	Low (stores path plus siblings)	High (stores all nodes at current level)
Completeness	No (may get stuck in infinite branches)	Yes (if branching factor finite)
Optimality	No (may find longer paths)	Yes for uniform step cost
Use Cases	When solution depth is unknown or solutions deep	When shortest path is required, or solution shallow

---

6. Pseudocode Examples

Depth-First Search (DFS):

pseudo

CopyEdit

```
DFS(node):
  if node is goal:
    return SUCCESS
  mark node as visited
  for each child in successors(node):
    if child not visited:
      result = DFS(child)
      if result == SUCCESS:
        return SUCCESS
  return FAILURE
```

Breadth-First Search (BFS):

pseudo

CopyEdit

```
BFS(initial_node):
  create queue Q
  enqueue initial_node into Q
  mark initial_node as visited
  while Q not empty:
    node = dequeue Q
    if node is goal:
      return SUCCESS
    for each child in successors(node):
      if child not visited:
        mark child as visited
        enqueue child into Q
  return FAILURE
```

---

7. Search Tree Example

- Consider a tree where node A connects to B and C.
- B connects to D and E; C connects to F and G.
- Goal node is G.

Search Strategy	Order of Node Expansion
DFS	A → B → D → E → C → F → G
BFS	A → B → C → D → E → F → G

---

8. Evaluation Metrics for Simple Search

Metric	Description
Completeness	Does the algorithm guarantee to find a solution if one exists?
Optimality	Does it find the least-cost solution?

Time Complexity	Number of nodes generated/expanded until solution is found.
Space Complexity	Memory required to store nodes (frontier + explored nodes).

---

9. Complexity Analysis

Algorithm	Time Complexity	Space Complexity	Notes
DFS	O(b^m) (b=branching factor, m=max depth)	O(bm)	Low memory but may be inefficient.
BFS	O(b^d) (d=depth of solution)	O(b^d)	Uses a lot of memory; finds shortest path.
UCS	O(b^(C*/ε)) (C* = cost of optimal path, ε = min step cost)	O(b^(C*/ε))	Expensive for large state spaces.

---

10. Strengths and Weaknesses of Simple Search

Strengths	Weaknesses
Simple to understand and implement	Inefficient for large or infinite spaces
Provides baseline for more advanced search	Uninformed: no domain knowledge used
DFS uses minimal memory	DFS not guaranteed to find shortest or any path in infinite spaces
BFS finds shortest path (if uniform cost)	BFS memory usage grows exponentially
UCS finds least cost path	UCS can be slow and memory-heavy

---

11. Real-World Applications

- **DFS:** Puzzle solving, maze navigation, scheduling where space is limited.
- **BFS:** Finding shortest paths in unweighted graphs, social networking analysis.
- **UCS:** GPS navigation, network routing where path costs vary.